

Communicating with the GO! Navigator BaO Selector

This document describes how external applications can interact with the GO! Navigator BaO Selector using the browser's `postMessage` API. The selector provides a robust, type-safe messaging protocol for bidirectional communication, enabling external parties to request actions and receive events from the selector.

Overview

Communication with the GO! Navigator BaO Selector is achieved by sending and receiving structured messages using `window.postMessage`. All messages follow a strict schema. The protocol distinguishes between:

- **Command Messages:** Sent to the selector to request actions or data.
- **Event Messages:** Sent from the selector to notify the host of state changes or actions.

Message Types

Command Messages (to selector)

Note: Commands are only accepted after you have received a `ready` event from the selector.

Type	Description	Data (if any)
<code>setSelection</code>	Set the selection in the selector	<code>{ selection: SimpleSelectionItem[] }</code>

Example: Set Selection

```
1 windowRef.postMessage(  
2   {  
3     type: 'setSelection',  
4     data: {  
5       selection: [  
6         /* ...SimpleSelectionItem objects... */  
7       ]  
8     }  
9   },  
10  '*',  
11 );
```

Event Messages (from selector)

Type	Description	Data (if any)
------	-------------	---------------

ready	Selector is initialized and ready	None
save	Selection has been saved	{ selection: Selection }
close	Selector tab is closing	None

Example: Event Message Structure

```

1 // Ready event
2 { type: 'ready' }
3
4 // Save event
5 { type: 'save', data: { selection: /* Selection */ } }
6
7 // Close event
8 { type: 'close' }
```

Integration Guide

1. Opening the Selector

The selector can be embedded either as a popup tab or as an `iframe`. Both use the exact same message protocol described above — nothing else changes.

Option A: Popup tab

Open the selector in a new window/tab and keep a reference:

```

1 const selectorWindow = window.open(
2   '/navigator-bao/selector/basisonderwijs?curriculumId=
3   <uuid>&curriculumItemId=<uuid>',
4   '<identifier_for_window>'
5 );
```

Option B: Iframe

Embed the same URL in an `iframe` and keep a reference to its `contentWindow`:

```

1 const iframe = document.createElement('iframe');
2 iframe.src = '/navigator-bao/selector/basisonderwijs?curriculumId=
3   <uuid>&curriculumItemId=<uuid>';
4 document.body.appendChild(iframe);
5 const selectorWindow = iframe.contentWindow;
```

Note: The URL can include query parameters to show a specific curriculum and/or a specific curriculum item, but this is optional.

2. Listening for Events

Add an event listener to handle messages from the selector:

```

1 window.addEventListener('message', event => {
```

```

2 // Optionally validate event.origin for security
3 const msg = event.data;
4 switch (msg.type) {
5   case 'ready':
6     // Selector is ready
7     break;
8   case 'save':
9     // Selection saved: msg.data.selection
10    break;
11   case 'close':
12     // Selector closed
13     break;
14 }
15 });

```

3. Sending Commands

Use `postMessage` to send commands to the selector:

```

1 selectorWindow.postMessage(
2   {
3     type: 'setSelection',
4     data: {
5       selection: [
6         /* ... */
7       ]
8     }
9   },
10  '*'
11 );

```

Example: Full Integration

```

1 const selectorWindow = window.open(
2   'https://g-o.smartschool.be/navigator-bao/selector/basisonderwijs',
3   'navigator_bao_selector'
4 );
5
6 // Listen for events
7 window.addEventListener('message', event => {
8   if (event.origin !== 'https://g-o.smartschool.be') return;
9   const msg = event.data;
10  if (msg.type === 'ready') {
11    // Selector is ready
12    console.info('ready');
13    // Send initial selection, if needed
14    selectorWindow?.postMessage(
15      {
16        type: 'setSelection',
17        data: {
18          selection: [
19            /* SimpleSelection */
20          ]
21        }
22      },
23      '*'
24    );
25  } else if (msg.type === 'save') {
26    // Handle saved selection
27    const selection = msg.data.selection;
28    console.info('saved', selection);
29  } else if (msg.type === 'close') {
30    // Selector closed

```

```
31     console.info('closed');
32   }
33 });
```

Message Schema Reference

All messages conform to the following schemas (written in TypeScript):

CommandMessage

```
1 type CommandMessage = { type: 'setSelection'; data: { selection:
  SimpleSelectedItem[] } };
```

What is SimpleSelectedItem?

SimpleSelectedItem is the type used to represent a single selection item in the selector.

Each item contains:

```
1 type SimpleSelectedItem = {
2   curriculumIdentifier: string; // Unique ID for the curriculum
3   curriculumItemIdentifier: string; // Unique ID for the curriculum
  item
4 };
```

This structure allows you to specify exactly which curriculum item is selected. When sending a selection, provide an array of these objects:

```
1 selectorWindow.postMessage(
2   {
3     type: 'setSelection',
4     data: {
5       selection: [
6         {
7           curriculumIdentifier: '<uuid>',
8           curriculumItemIdentifier: '<uuid>'
9         }
10        // ...more items
11      ]
12    }
13  },
14  '*')
15 );
```

EventMessage

```
1 type EventMessage =
2   | { type: 'ready' }
3   | {
4     type: 'save';
5     data: {
6       selection?: Selection;
7     };
8   }
9   | { type: 'close' };
```

Selection

```
1 type Selection = [string, NavigatorStructureItem][]; // [[<uuid>,
NavigatorStructureItem], ...];
```

NavigatorStructureItem

```
1 type NavigatorStructureItem = {
2   curriculumItem: NavigatorItem; // The curriculum item data
3   children: NavigatorStructureItem[]; // Nested children items
4 };
```

NavigatorItem

```
1 type NavigatorItem = {
2   metadata: NavigatorMetadata;
3   description?: string;
4   identifier: string; // uuid
5   platformId: number;
6   source: string; // the source of the item, e.g. "navigator-bao"
7   text: string;
8   curriculumPlatformId: number;
9   curriculumIdentifier: string; // Unique ID for the curriculum
10  category: string; // e.g. "information" | "goal";
11  type?: string; // e.g. "title", "defaultGoal"
12  breadcrumbs?: Breadcrumb[];
13 };
```

NavigatorMetadata

```
1 type NavigatorMetadata = {
2   prefix?: string;
3   icon?: string | null;
4   level?: number;
5   linkedCurriculumItems?: NavigatorLinkedItem[];
6   labels?: CurriculumLabel;
7   informationUrls?: InformationUrl;
8   minimumGoalType?: string; // e.g. "K0", "L4", "L6"
9   groupId?: string; // uuid
10 };
```

NavigatorLinkedItem

```
1 type NavigatorLinkedItem = {
2   curriculumItem: NavigatorItem;
3   type: string; // e.g. "minimum", "related", "sequential"
4 };
```

CurriculumLabel

```
1 type CurriculumLabel = {
2   id: string; // uuid,
3   platformId?: number;
4   text: string;
5   color?: string;
6   isVisible?: boolean;
7   type: string; // e.g. "knowledge_processing", "learning_path"
8 };
```

InformationUrl

```
1 type InformationUrl = {
2   type: string; // e.g. "fiche"
3   url: string;
4 };
```

Breadcrumb

```
1 type Breadcrumb = {  
2   identifier: string;  
3   text: string;  
4   type: string; // e.g. "curriculum", "curriculum_item"  
5 };
```